# Access Console

Client-Side Cryptographic Account Registration & Management System

`v1.0.0 Single-File HTML5` `Web Crypto API (SHA-256)` `Vanilla JS ES6+ (No Frameworks)` `Persistent LocalStorage`

System README & Spec

`access_console_users_v1`
16-Byte CSPRNG Salt
Browser Sandboxed

## 1. Project Overview & Architecture

**Access Console** is a high-performance, single-file administrative interface developed using vanilla web technologies (HTML5, modern CSS3, and ES6+ JavaScript)[cite: 1]. It serves as a secure client-side user registration portal, cryptographic demonstration engine, and local credential management console[cite: 1]. The entire application runs natively in any standard web browser without requiring Node.js runtime environments, bundlers (Webpack, Vite), or external third-party JavaScript dependencies[cite: 1].

The system demonstrates a **zero-plaintext storage paradigm**[cite: 1]: user passwords entered into the interface are never retained in plaintext memory or committed unhashed to the storage medium[cite: 1]. Instead, each password is merged with an isolated, cryptographically secure 16-byte pseudo-random salt and processed through the browser's native hardware-accelerated `window.crypto.subtle.digest` engine before persistence[cite: 1].

| LAYER / SUBSYSTEM | TECHNOLOGY APPLIED | CORE FUNCTIONAL RESPONSIBILITY |
|---|---|---|
| **Presentation Layer** | HTML5 & CSS3 Variables / Keyframe FX | High-contrast cyber-styled theme, animated scanlines, responsive mobile card transformation[cite: 1]. |
| **Cryptographic Engine** | W3C Web Cryptography API (SubtleCrypto) | Non-blocking SHA-256 hashing calculations and 128-bit CSPRNG salt generation[cite: 1]. |
| **State & Validation** | Vanilla Event-Driven Reactive JS | Form debouncing, race-condition mitigation via monotonic IDs, sorting, search, and toasts[cite: 1]. |
| **Persistence Engine** | HTML5 LocalStorage API | JSON serialization and atomic state persistence across browser restarts and reloads[cite: 1]. |

## 2. How to Run the Project

The codebase is contained entirely within a single standalone file (`access-console.html`)[cite: 1]. To execute and test the application, follow either method below:

### Method A: Direct Browser Execution

1. Download or clone `access-console.html` to your machine[cite: 1].
2. Double-click the file or open it in any modern browser (Chrome, Edge, Firefox, Safari, Brave)[cite: 1].
3. The application initializes instantly with full cryptographic capabilities[cite: 1].

### Method B: Local Web Server (Recommended)

For strict cross-origin isolation or automated CI testing:
- **Python:** `python -m http.server 8000`
- **Node.js:** `npx serve .` or `npx live-server`
- Open `http://localhost:8000/access-console.html` in your browser.

**Storage Persistence Notice**

When previewed inside sandboxed iframe containers, browser privacy controls may disable `localStorage` write access[cite: 1]. Running the file directly from disk or via a standard localhost web server guarantees full read/write persistence across page refreshes[cite: 1].

## 3. Core Features Implemented

- **Account Registration Form:** Collects user credentials (Username, Email, and Password) with real-time field validation and constraints (minimum 6-character passwords, standard email regex formatting)[cite: 1].
- **Cryptographic Salting:** Every password is automatically paired with a unique, cryptographically random 16-byte salt generated via `crypto.getRandomValues` to defend against dictionary and rainbow-table attacks[cite: 1].
- **Client-Side SHA-256 Hashing:** Performs asynchronous cryptographic digest generation using the browser's native Web Crypto API prior to storing the credential[cite: 1].
- **Records Management Dashboard:** An administrative ledger displaying registration sequence index numbers, usernames, emails, truncated hash previews, and deletion controls[cite: 1].
- **Persistent Local Storage:** Serializes and syncs user objects to `localStorage` under `access_console_users_v1`, enabling persistent sessions across browser restarts[cite: 1].
- **Record Deletion Pipeline:** Allows operators to cleanly purge individual accounts from the active ledger, triggering automatic row animations and storage re-indexing[cite: 1].

## 4. Additional Features Added Beyond Baseline Requirements

The console includes several sophisticated architectural enhancements to deliver a responsive, production-quality user experience:

| FEATURE | TECHNICAL IMPLEMENTATION | USER & OPERATIONAL BENEFIT |
|---|---|---|
| **Live Hash Ticker** | Monotonic request-ID tracking paired with dynamic `crypto.subtle` digest computation[cite: 1]. | Visualizes SHA-256 calculation live as the user types; prevents race conditions during rapid typing[cite: 1]. |
| **Password Strength Meter** | Heuristic scoring evaluating length (≥6, ≥10), mixed casing, numerals, and symbols[cite: 1]. | Interactive 4-tier visual progress bar (Weak → Strong) providing instant complexity guidance[cite: 1]. |
| **Debounced Collision Checks** | 350ms non-blocking input debouncers paired with dynamic SVG spinner/check icons[cite: 1]. | Real-time validation detecting duplicate usernames or emails against existing records without UI stutter[cite: 1]. |
| **Multi-Column Sorting** | Dual-direction comparator sort algorithms for Username and Email with animated carets[cite: 1]. | Enables rapid alphabetical sorting and organization of large credential registries[cite: 1]. |
| **Instant Filter & Hotkey** | Case-insensitive substring search bound to the forward-slash (`/`) keyboard shortcut[cite: 1]. | Allows operators to instantly locate accounts; pressing `/` auto-focuses the search bar[cite: 1]. |
| **Undo Toast Recovery** | Non-blocking notification stack with a 5-second closure restoration timer[cite: 1]. | Protects against accidental deletions by allowing instant one-click restoration of purged records[cite: 1]. |
| **One-Click Clipboard Export** | Asynchronous `navigator.clipboard.writeText` with SVG checkmark confirmation[cite: 1]. | Instantly copies the full `salt:hash` string to the clipboard for auditing or debugging[cite: 1]. |
| **Visibility Switcher** | Input type toggling between `password` and `text` with custom SVG eye vectors[cite: 1]. | Improves credential input accuracy while preserving on-screen confidentiality[cite: 1]. |
| **Responsive Card Reflow** | Media query breakpoint converting data table rows into stacked cards via `data-label`[cite: 1]. | Provides a native mobile experience on smartphones and small viewports without horizontal clipping[cite: 1]. |

## 5. Security Architecture & Cryptographic Design

The console follows strict cryptographic implementation standards:

```
// 1. Generate 16 bytes (128 bits) of cryptographically secure pseudo-random entropy
function randomSaltHex() {
  const saltBytes = crypto.getRandomValues(new Uint8Array(16));
  return bufferToHex(saltBytes.buffer);
}

// 2. Compute non-blocking SHA-256 digest using the browser's hardware crypto engine
async function hashPassword(plainPassword) {
  const salt = randomSaltHex();
  const buffer = await crypto.subtle.digest(
    "SHA-256",
    new TextEncoder().encode(salt + plainPassword)
  );
  return { salt: salt, hash: bufferToHex(buffer) };
}
```

**Rainbow Table Immunity**

Storing bare `SHA-256(password)` hashes leaves systems open to reverse rainbow-table attacks. Prepending a random 128-bit salt guarantees every user generates a distinct digest even if identical passwords are selected.

**Zero Plaintext Residue**

Plaintext strings exist transiently in RAM during form processing. Once passed to the digest encoder, memory pointers are released and only `{ salt, hash }` tuples are stored in `localStorage`[cite: 1].

## 6. Concepts Learned & Engineering Takeaways

- **W3C Web Cryptography API (`window.crypto`):** Acquired in-depth experience utilizing modern browser cryptographic primitives (`crypto.subtle.digest` and `crypto.getRandomValues`) to compute secure SHA-256 digests asynchronously via Promises[cite: 1].
- **Handling Asynchronous UI Race Conditions:** Solved an essential asynchronous UI challenge where fast typing caused earlier hash computations to resolve *after* later keystrokes[cite: 1]. Solved this by introducing a monotonic `hashRequestId` counter, discarding superseded promises[cite: 1].
- **Non-Blocking Event Debouncing:** Implemented timer-driven debouncing (`setTimeout` / `clearTimeout`) to pause resource-intensive duplicate collision checks during active keystroke bursts[cite: 1].
- **Framework-Free Declarative DOM Management:** Structured clean application state (active tab, sort order, search query, records list) and projected state changes to the DOM via procedural table rendering without the overhead of external libraries like React or Vue[cite: 1].
- **CSS Grid, Flexbox, & Adaptive Layouts:** Crafted an accessible, animated interface leveraging CSS variables, keyframe animations, media query transitions, and pseudo-element data labeling for small viewports[cite: 1].

## 7. Extended Modules & Production Roadmap

### 7.1 Client-Side Data Portability (CSV & JSON Export)

Enables administrators to export the stored user ledger into structured JSON or spreadsheet-friendly CSV files directly through client-side data URIs:

```
function exportToCSV() {
  const headers = ["ID", "Username", "Email", "Salt", "Hash"];
  const rows = users.map(u => [
    `"${u.id}"`, `"${u.username.replace(/"/g, '""')}"`, `"${u.email.replace(/"/g, '""')}"`, `"$
{u.salt}"`, `"${u.hash}"`
  ]);
  const csvContent = "data:text/csv;charset=utf-8," + [headers.join(","), ...rows.map(r =>
r.join(","))].join("\n");
  const anchor = document.createElement("a"); anchor.href = encodeURI(csvContent);
  anchor.download = `access_console_export_${Date.now()}.csv`; anchor.click(); anchor.remove();
}
```

### 7.2 Account Authentication & Verification Protocol

Provides user login verification by fetching the saved salt, hashing candidate passwords, and evaluating matches:

```
async function verifyCredentials(identifier, candidatePassword) {
  const cleanId = identifier.trim().toLowerCase();
  const record = users.find(u => u.username.toLowerCase() === cleanId || u.email.toLowerCase() ===
cleanId);
  if (!record) return { success: false, message: "User not found." };
  const candidateHash = await sha256Hex(record.salt + candidatePassword);
  return candidateHash === record.hash
    ? { success: true, user: { id: record.id, username: record.username } }
    : { success: false, message: "Invalid password." };
}
```

### 7.3 Server-Side Migration Strategy (Node.js / Express Microservice)

For multi-tenant or enterprise scenarios, the client-side architecture transitions cleanly to a centralized backend:

- **Server-Side Password Hashing:** Upgrade to memory-hard hashing algorithms such as **Argon2id** or **bcrypt** (work factor ≥ 12) to resist ASIC and GPU brute-force attacks.
- **Transport Layer Security:** Enforce HTTPS / TLS 1.3 encryption on all transmission channels.
- **Stateless Session Tokens:** Issue signed JWTs or HTTP-only Secure SameSite session cookies upon successful verification.

## 8. Repository File Structure

```
access-console/
├── access-console.html          # Complete single-file application (UI, styles, logic)
├── README.md                    # Markdown documentation for code repositories
└── access_console_readme.pdf    # Compiled technical architecture specification & manual
```

**Access Console Documentation** • Designed with Vanilla Web Standards • Compiled for Production Specification